

PRODUCTIVE FAILURE REPORT

Naive Conversational Memory Baseline

Conversational Memory Intelligence System

A measured baseline for understanding where similarity-only memory retrieval breaks down.

Experiment property	Configuration
Baseline scope	Deliberately simple, similarity-only conversational memory
Workload	Fixed six-case workload
Embedding model	sentence-transformers/all-mpnet-base-v2
Retrieval	Semantic similarity, top-k = 10
Memory store	Single shared collection
Similarity threshold	None
Random seed	10
Evidence	Runnable baseline + protocol + machine-readable results + error examples

This report records what the experiment actually established, including results that did not match the initial hypothesis.

1. Hypothesis

I expected the naive design to work reasonably well for straightforward memory lookup: extract candidate memories, store all of them, and retrieve the most semantically similar memories for a new query. I also expected it to become unreliable when memories conflict, change over time, belong to different users, contain sensitive information, or when there is no relevant memory.

The main assumption being tested was simple: semantic similarity is enough to reconstruct the context needed for the current request.

2. Baseline Design

The baseline was intentionally kept simple rather than being artificially made to fail. Every extracted memory is stored. Each memory is embedded with sentence-transformers/all-mpnet-base-v2. A query is embedded and compared with stored memories using semantic similarity, and the top 10 results are returned.

- No write-side importance or admission judgement.
- No recency or temporal-validity weighting.
- No conflict or supersession handling.
- No decay or consolidation.
- No deletion or forget enforcement.
- No token-budget-aware context selection.
- No user or tenant filtering during retrieval.
- No similarity threshold or explicit no-memory state.

These omissions are deliberate. The baseline is meant to be a fair reference point: failures should come from the limitations of a plausible simple design, not from code that was intentionally made defective.

3. Workload and Experimental Configuration

The workload was defined before the benchmark run and contains six targeted cases covering the main failure conditions expected from a conversational memory system.

Case	Failure condition tested	What the case is designed to expose
1	Irrelevant + contradictory memories	Semantic similarity may retrieve conflicting history or plausible distractors.
2	Changing state over time	Old and new decisions may both remain retrievable.
3	Longer memory pool / historical pollution	A growing memory pool may return multiple historical versions of the same topic.
4	Multiple users	A shared store without user filtering can return another user's memory.
5	Sensitive information	Stored sensitive content can remain retrievable after a request not to retain it.

6	Cold start / no relevant memory	Top-k retrieval can return unrelated memories when no useful memory exists.
---	---------------------------------	---

Parameter	Configuration
Embedding model	sentence-transformers/all-mpnet-base-v2
Retrieval	Semantic similarity
Top-k	10
Dataset	Fixed six-case workload, real + synthetic cases
Random seed	10
Memory store	Single shared collection
Similarity threshold	None
Context construction	Retrieved memories + actual evaluation query

4. Metrics and What Was Measured

The benchmark records case-level retrieval behavior, failure categories, and retrieval latency. The experiment does not claim measurements that were not actually collected.

Metric	Status	Interpretation
Retrieval / failure evidence	Measured	Per-case evaluator records whether the targeted failure condition occurred.
Latency	Measured	One recorded timing per case; useful for baseline reference, not a production benchmark.
End-to-end response quality	Not measured	No LLM answer-quality benchmark was run.
Prompt-token usage	Not measured	The experiment did not perform token accounting.
Storage growth	Not measured	No longitudinal storage-growth experiment was run.
Monetary cost	Not measured	No production cost model was evaluated.
Qualitative observations	Recorded	Negative results, duplicate memories, and context-construction issues were inspected.

5. Results

All six cases produced at least one observed failure under their evaluators. Case 1 is especially useful because one predicted failure was not observed: the apartment-memory retrieval did not occur. The contradiction failure did occur.

Case	Memories	Retrieved	Latency (ms)	Observed failure
1	18	10	87.830	Contradictory memories
2	3	3	72.706	Old + new decisions
3	50	10	61.810	Multiple historical decisions
4	17	10	75.346	Cross-user retrieval
5	17	10	59.942	Sensitive memory retrievable
6	17	10	74.902	Unrelated cold-start retrieval

Across the six single-run case timings, the median latency was 73.804 ms and the descriptive 95th percentile was 84.709 ms. These should not be interpreted as production p50/p95 benchmarks: each case has only one recorded run, and model-loading and runtime effects were not characterized.

5.1 Case-level evidence

Case 1: The retriever returned both the old `vector_store.pkl` decision and the later FAISS decision. The predicted apartment-related distractor was not retrieved.

Case 2: All three stored memories were retrieved, including both the old storage decision and the newer FAISS decision.

Case 3: All three historical architecture signals were retrieved from a 50-memory pool. The same filler memory also appeared three times with the same score, exposing a separate duplication-quality issue.

Case 4: Both user A's expected memory and another user's similarly worded memory were returned from the shared collection.

Case 5: The API-key memory and the explicit forget-request memory were both retrieved.

Case 6: Ten unrelated memories were returned for a query about a topic for which the memory store had no relevant information.

6. Failure Taxonomy

Observed failure	Violated assumption	System / user impact
Case 1: contradictory memories retrieved	Similarity finds related memories, but does not encode which architectural decision is current.	The model can receive mutually inconsistent historical states.
Case 2: old and new decisions both retrieved	A newer state does not automatically supersede an older, semantically similar state.	A current-state question can be answered using obsolete information.

Case 3: three historical architectural decisions retrieved	Similarity alone does not resolve validity when several versions of the same topic exist.	Context can contain multiple versions of one decision, increasing ambiguity and memory noise.
Case 4: another user's memory retrieved	All users share one collection and user identity is ignored during retrieval.	Creates a correctness problem and a potential cross-user information-leakage path.
Case 5: sensitive memory remains retrievable	Every extracted memory is stored without admission or deletion enforcement.	Sensitive information can remain available after the user has asked not to retain it.
Case 6: unrelated memories retrieved	Top-k retrieval always returns results because there is no relevance threshold or no-memory state.	The system can inject false context instead of recognizing that it has no useful memory.

7. What the Experiment Actually Established

The strongest result is not simply that retrieval is bad. In several cases the retriever found semantically relevant memories. The problem is that relevance is not the same as validity.

- Cases 1–3 show that several historical states can all be relevant to the same query even though only one should normally determine the current answer.
- Case 1 falsified part of the initial hypothesis: the predicted apartment-memory confusion was not observed in the actual embedding run.
- Case 3 shows historical-state pollution as the memory pool grows. It does not prove actual context-window overflow because token usage was not measured.
- Case 2 uses a changing architectural decision rather than a personal preference. It therefore tests the broader changing-state and supersession problem.
- Case 4 demonstrates that semantic relevance is not an adequate isolation boundary.
- Case 5 demonstrates that storing a sensitive memory is not enough; retention and deletion semantics have to affect what can later be retrieved.
- Case 6 demonstrates the need for an explicit way to decide that no stored memory is useful.

7.1 Additional observation: duplicate memories

Case 3 contained the same filler memory three times with the same score. This did not change the primary failure because all three architectural signals were still retrieved, but it exposes a separate data-quality concern: a memory system should not allow accidental duplication to consume retrieval capacity.

7.2 Evaluation-output correction

During the cleanup pass, the generated [Current Conversation] field was corrected to use the actual evaluation query rather than the last scripted conversation turn. This was an evidence/reporting correction, not a retrieval improvement. The naive retrieval behavior itself was left unchanged.

8. Proposed Response for the Next Design

The following are design hypotheses derived from the observed failures. They are not treated as automatically correct; each should be validated before becoming part of the architecture.

Observed failure	Design response to investigate	Expected benefit	Trade-off / risk
Contradictory or superseded memories	Represent temporal validity and explicit relationships such as supersedes, replaces, deprecated, or current.	Retrieve relevant memories while resolving which state is currently valid.	More complex representation, update logic, and evaluation.
Unrelated retrieval / cold start	Add a calibrated relevance gate or explicit no-relevant-memory state before context injection.	Avoid injecting low-value memories when similarity is insufficient.	Thresholds can create false negatives and require validation.
Cross-user retrieval	Make user/tenant identity a retrieval constraint rather than metadata that is ignored.	Prevent another user's memories from entering the candidate set.	Requires authorization/isolation checks and tenant-aware indexing or filtering.
Sensitive memory retained	Add admission policy plus explicit deletion/forget semantics affecting both source storage and retrieval.	Prevent sensitive data from entering long-term memory and make deletion meaningful.	Policy complexity and consistency requirements.
Historical/context pollution	Add context selection and token budgeting instead of blindly injecting every top-k result.	Bound memory contribution and prioritize useful current context.	Requires token accounting, ranking policy, and additional evaluation.

9. Falsification Criteria

These proposed responses should be rejected, simplified, or revised if controlled experiments show that they do not improve the relevant failure without unacceptable regressions.

- Conflict/supersession handling is unnecessary if a fixed workload shows that similarity-only retrieval consistently selects the valid current state without exposing obsolete states.
- A relevance gate is unnecessary if a calibrated no-memory test shows that the baseline can already distinguish relevant from unrelated queries at an acceptable operating point.
- Tenant filtering is inadequate if adversarial cross-user tests still return another user's memory.
- Admission/deletion controls are inadequate if a forgotten sensitive memory remains retrievable within the documented consistency window.
- Context budgeting is inadequate if it reduces useful recall enough to outweigh its benefit, or if measured token usage shows that the baseline does not approach the configured context constraint under the intended workload.

10. Limitations

- The latency figures are based on one run per case and are not representative production p50/p95 benchmarks.
- End-to-end LLM response quality was not measured.
- Prompt-token usage was not measured, so actual context-window overflow was not demonstrated.
- Storage growth over time was not measured.
- Monetary cost was not measured.

- The workload is intentionally small and targeted; it is evidence for specific failure modes, not a general benchmark of all conversational memory systems.
- The experiment uses a single embedding model and similarity signal, so the findings should be validated again when the retrieval architecture changes.

11. Reproducibility and Evidence Trail

The experiment is implemented under `experiments/naive_baseline/`. The repository contains the runnable baseline, fixed workload cases, evaluators, and machine-readable outputs used for this report.

Artifact	Purpose
<code>experiments/naive_baseline/</code>	Runnable baseline implementation and workload execution code.
<code>experiments/baseline_protocol.md</code>	Defines the experiment configuration, workload, and evaluation protocol.
<code>experiments/baseline_results.csv</code>	Machine-readable case-level results and latency measurements.
<code>experiments/error_examples.jsonl</code>	Per-case retrieved memories, evaluator outputs, and assembled context.

The final rerun was performed after correcting the context-construction issue described above. The corrected machine-readable outputs are the evidence used for this report. The baseline retrieval logic was not modified to improve its results.

12. Conclusion

The naive baseline gives a clear starting point for the next stage of the system. Across the six targeted cases, each failure condition was represented by an observed failure, while one additional predicted failure was tested and not confirmed.

The main lesson is that semantic retrieval by itself is not enough for conversational memory. A useful system needs mechanisms for deciding what should be stored, which memories belong to the current user and situation, which state is current, when memory should be removed, and when no memory should be retrieved at all.

These results do not prove that any one replacement architecture is correct. They define the problems that the next architecture must address and provide a fixed set of failures against which those changes can be evaluated.

Appendix A — Representative Retrieved Memories

A few representative outputs are included here to make the failure evidence concrete without reproducing the full machine-readable error file.

Case	Representative evidence
1	Old storage decision and later FAISS decision were both retrieved for a storage-architecture question.

2	The old vector_store.pkl decision ranked above the current FAISS decision for a current-state query.
3	Three architectural states were retrieved from a 50-memory pool; duplicate filler memories also occupied retrieval slots.
4	Flask and FastAPI memories from different users were both retrieved for a deployment-framework question.
5	The temporary API key remained retrievable even after a memory explicitly stated it should not be remembered.
6	Ten unrelated personal/project memories were returned for a Japan itinerary query with no relevant stored memory.

Primary evidence: experiments/baseline_protocol.md, experiments/baseline_results.csv, experiments/error_examples.jsonl, and experiments/naive_baseline/.